

# Real and Complex FFT Implementation using CMSIS

## Electronics Project 4 (ET-PM4): CW-Doppler Radar Sensor

### Abstract

Nowadays, the Fast Fourier Transformation (FFT) functions are typically no longer written oneself. Instead SDK / library functions are used, which are already optimised for the corresponding microcontroller platform. In this document, the “CMSIS DSP Software Library Version 1.5.2” is used.

In the PM4 Project the evaluation board "STM32F429I-DISC1" is used, which is based on a Cortex M4F. The "F" means that the microcontroller contains a floating-point unit (FPU). Accordingly, this document uses the floating-point FFT implementation.

In addition, there is a small section on how the data from the ADC must be converted in order to use it as a time signal input value for the FFT.

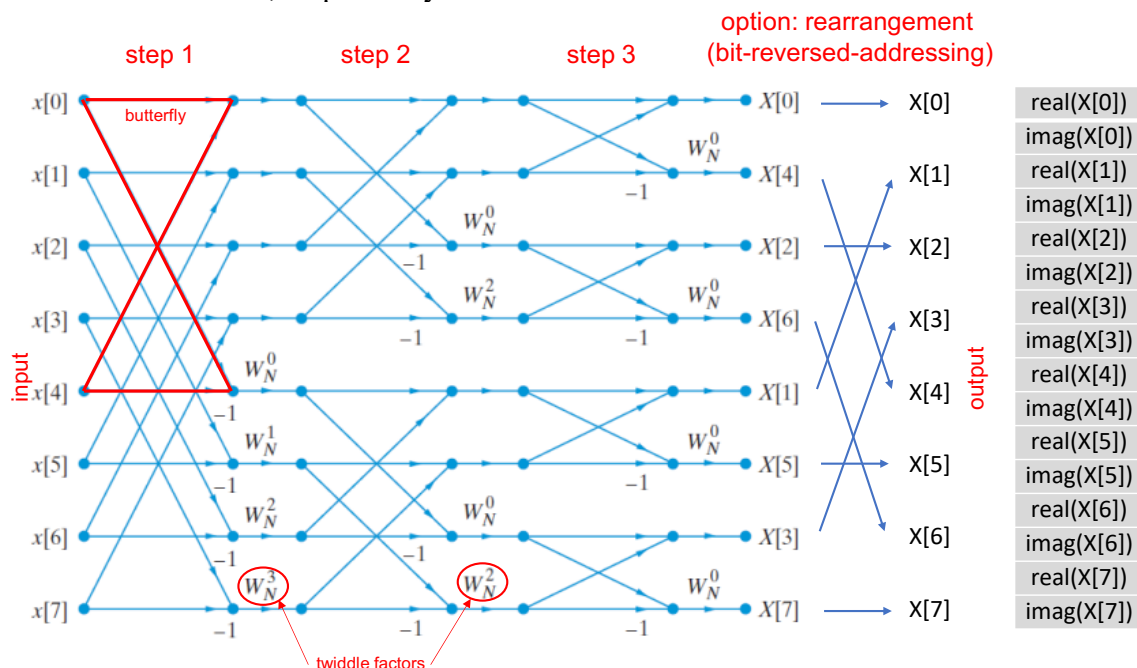
### Table of Content

|                                                                |   |
|----------------------------------------------------------------|---|
| Abstract .....                                                 | 1 |
| 1. Review of the Fast Fourier Transform (FFT) .....            | 2 |
| 2. CMSIS Implementation .....                                  | 2 |
| 2.1. FFT Strukt Initialisation .....                           | 5 |
| 2.2. FFT Preparation and Function Call .....                   | 6 |
| 2.2.1. Type Cast from ADC Dual Mode Samples .....              | 6 |
| 2.2.2. Optional Zero Padding.....                              | 6 |
| 2.2.3. FFT Function Call .....                                 | 6 |
| 2.2.4. Test Signal.....                                        | 7 |
| 2.3. Some useful CMSIS Functions.....                          | 7 |
| 2.4. Some possible errors.....                                 | 8 |
| 2.4.1. __FPU_PRESENT.....                                      | 8 |
| 2.4.2. Hard Fault when calling <code>arm_cfft_f32</code> ..... | 8 |

# 1. Review of the Fast Fourier Transform (FFT)

The Fast Fourier Transform (FFT) algorithm computes the Discrete Fourier Transform (DFT) of a sequence in an efficient way. It reduces the complexity of computing the DFT from  $O(N^2)$  to  $O(N \cdot \log_2(N))$ , where  $N$  is the data length of the input sequence. This is achieved by taking advantage of redundancies and symmetry as well as pre calculated so-called Twiddle Factors  $W_N = e^{-j2\pi/N}$ .

As an example, the decimation-in-frequency radix 2 FFT-algorithm is shown in Figure 1 for  $N=8$  input samples  $x[n]$ . In the flow-graph, an arrow represents a multiplication (by 1 if nothing else is noted) and a node represents an addition. The  $N=8$  complex DFT spectrum-values  $X[m]$  are computed in  $\log_2(N)=3$  steps. In each step,  $N/2=4$  so called Butterfly's have to be calculated, whereby 2 values are overwritten in the memory in each case. The whole FFT-algorithm can be computed «in-place» with a single array with  $2 \cdot N=16$  memory cells, where the real and imaginary parts of the computed values are stored in the memory cells at even and odd addresses, respectively.



**Figure 1:** Decimation-in-frequency radix 2 8-point FFT-algorithm, see DSV1 script. With the decimation-in-frequency FFT-algorithm, the spectrum-values  $X[m]$  are not in ascending order. However, most FFT library functions take over the correct ordering as an option with the bit-reversed-addressing operation. Using this operation, the binary index (e.g. 001 or “1”) is read right-to-left and the result (e.g. 100 or “4”) is used as the address in the output array.

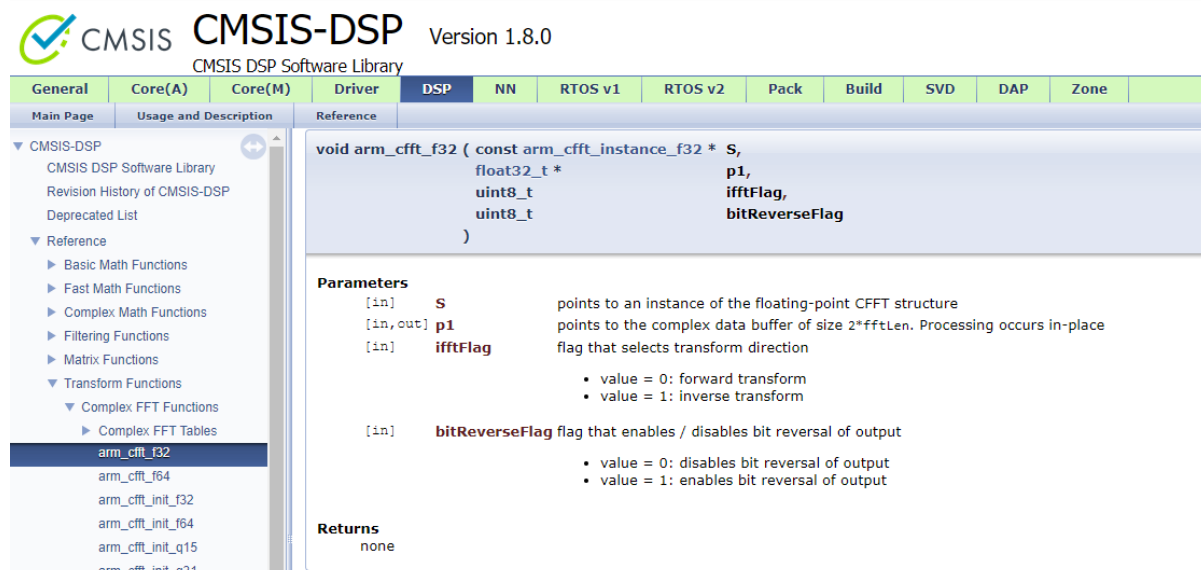
The IDFT and the DFT are identical except for a normalization factor and the sign in the twiddle factors. Therefore, the IFFT is integrated in most FFT library functions as an option.

## 2. CMSIS Implementation

The Cortex Microcontroller Software Interface Standard (CMSIS) is a vendor-independent hardware abstraction layer for microcontrollers that are based on Arm Cortex processors.

An overview over all available DSP (digital signal processing) library functions can be found in [1], [CMSIS-DSP Software Library](#). Under Reference -> Transform Functions there are lists of complex and real FFT library functions for different data types and, additionally, a description at the end of the pages.

The FFT-computation and the FFT-output are typically complex-valued. However, there are so-called real FFT functions, which are optimized for a single real-valued input signal. In the PM4 module the ordinary complex FFT is to be preferred, because the CFFT supports complex-valued input signals too. Moreover, the 32 bit floating-point data type F32 should be preferred because the processing of floating-point numbers takes only marginally longer than the processing of integer numbers thanks to the microcontroller's floating point unit [2] and, additionally, because scaling is no longer necessary. For the interested reader, FFT-cycle-counts for various FFT block lengths N and data types are shown on pages 10 and 11 in [3]. In PM4, the CMSIS-DSP-Library function `arm_cfft_f32` can be used, see Figure 2.



**Figure 2:** Documentation of the CMSIS-DSP-Library function `arm_cfft_f32` [1]. `p1` is a pointer to the complex data buffer with  $2 \cdot N$  array-elements of data type `float32_t`. Thus, the input of the function is the address (unary operator `&`) of the first element of the array.

At the function call, the array contains the samples of the 2 radar sensor outputs, and after leaving the function, the array contains the spectrum values. The input samples are overwritten.

The flags can be defined as constants (after the `#include` section)

```
#define IFFT_FLAG 0
#define BIT_REVERSE_FLAG 1
```

since in PM4 only the FFT with output sorted in ascending order is required.

Finally, the struct `arm_cfft_instance_f32` containing the FFT-length, tables for the twiddle-factors and bit reversed addressing etc. must be initialized for the library function `arm_cfft_f32`, see [CMSIS-DSP Software Library](#) under Data Structures and Figure 3.

| General         | Core(A)               | Core(M) | Driver    | DSP | NN | RTOS v1 | RTOS v2 | Pack | Build | SVI |
|-----------------|-----------------------|---------|-----------|-----|----|---------|---------|------|-------|-----|
| Main Page       | Usage and Description |         | Reference |     |    |         |         |      |       |     |
| Data Structures | Data Fields           |         |           |     |    |         |         |      |       |     |

▶ Reference

▼ Data Structures

- train
  - ▶ arm\_bilinear\_interp\_instance\_f32
  - ▶ arm\_bilinear\_interp\_instance\_q15
  - ▶ arm\_bilinear\_interp\_instance\_q31
  - ▶ arm\_bilinear\_interp\_instance\_q7
  - ▶ arm\_biquad\_cas\_df1\_32x64\_ins\_q31
  - ▶ arm\_biquad\_cascade\_df2T\_instance\_f32
  - ▶ arm\_biquad\_cascade\_df2T\_instance\_q15
  - ▶ arm\_biquad\_cascade\_df2T\_instance\_q31
  - ▶ arm\_biquad\_cascade\_stereo\_df2T\_instance\_f32
  - ▶ arm\_biquad\_cascade\_stereo\_df2T\_instance\_q15
  - ▶ arm\_biquad\_cascade\_stereo\_df2T\_instance\_q31
  - ▶ arm\_cfft\_instance\_f32

### arm\_cfft\_instance\_f32 Struct Reference

Instance structure for the floating-point CFFT/CIFFT function. [More...](#)

#### Data Fields

|                   |              |
|-------------------|--------------|
| uint16_t          | fftLen       |
| const float32_t * | pTwiddle     |
| const uint16_t *  | pBitRevTable |
| uint16_t          | bitRevLength |

#### Description

**Examples:**

**Figure 3:** Instance structure for floating-point CFFT function [1].

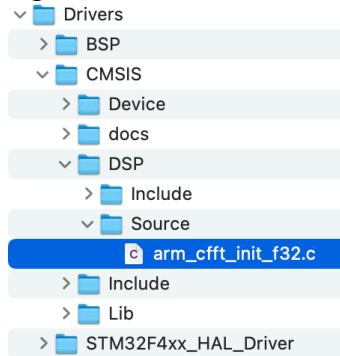
## 2.1. FFT Strukt Initialisation

As a first step in the firmware, you must initialize the above-mentioned structure once:

`arm_cfft_instance_f32` for the complex valued input or

`arm_rfft_fast_instance_f32` for the real valued input FFT. The structure variable should be static allocated so that it can be used later for all the FFT calls again and again.

The ADC firmware examples from PM3 (on the PM3 Pool), which you can use for the PM4, are not yet updated to the latest CMSIS version. The used version 1.5.2 in the examples does not yet have an init function for the `arm_cfft_instance_f32` structure. You can initialize the structure yourself or copy the file "arm\_cfft\_init\_f32.c" from the FFT folder in the PM4 Pool folder to the following location and include it in the project:



**Figure 4:** ARM CFFT INIT F32 function

Of course you can also update to the newest CMSIS Library, but be aware that you have to adjust quite a lot and therefore make a backup beforehand.

If you use the init function the call will look something like this for the Complex FFT Function:

```
status = arm_cfft_init_f32(&cfft_instance, DOPPLER_FFT_SIZE);
```

or for the Real FFT Function

```
status = arm_rfft_fast_init_f32(&rfft_instance, DOPPLER_FFT_SIZE);
```

## 2.2. FFT Preparation and Function Call

### 2.2.1. Type Cast from ADC Dual Mode Samples

When the ADC is operated in dual mode, both converted data from ADC1 and ADC2 are packed into a 32-bit register in this way:

```
ADC_CDR[31:0] = ADC2_DR[15:0] | ADC1_DR[15:0]
```

The samples must now first be separated, and can be cast as float at the same time. If you configure the dma suitably, you will get just as many values as you need for the FFT.

Accordingly you can cast the values in a for loop and copy them into the inout buffer for the fft.

```
...
float32_t sample_adc1;
float32_t sample_adc2;
for (n = 0; n < meas_samples_n; n++)
{
    sample_adc1 = (float32_t)(adc_dual_mode_samples[n] & 0x0000FFFF);
    sample_adc2 = (float32_t)((adc_dual_mode_samples[n] >> 16) & 0x0000FFFF);
    // todo : fill into the inout buffer for the FFT
    // format : { real[0], imag[0], real[1], imag[1], real[2], imag[2] ... }
}
```

Depending on which ADC channel you take as real or imaginary part, you have to switch or adjust a sign, so that in the end the sign of the frequency matches the expectations.

### 2.2.2. Optional Zero Padding

If you have fewer adc samples than the chosen FFT size, you can fill the remaining values with zero padding, for example.

### 2.2.3. FFT Function Call

The Complex FFT function call will look something like this:

```
arm_cfft_f32(&cfft_instance, cfft_inout, IFFT_FLAG, DO_BIT_REVERSE);
```

Where when the function is called, the array `cfft_inout` contains the complex time signal, which is then overwritten with the complex spectral values by the inplace calculation.

C pointer refresher:

```
float32_t cfft_inout[2*FFT_SIZE]
```

Since `cfft_inout` is an array, `cfft_inout` is interpreted as a pointer and points to the first element in the array. The same result you get if you write `&cfft_inout[0]`.

If you use the Real FFT function which is optimized for real input signals, then the call will look something like this:

```
arm_rfft_fast_f32(&rfft_instance, fft_in, fft_out, IFFT_FLAG);
```

`fft_in` and `fft_out` are arrays with the length `FFT_SIZE`.

Where when the function is called, the array `fft_in` contains the “real” time signal, and in the `fft_out` array the calculated complex spectral values will be stored.

Only `FFT_SIZE/2` complex spectral values will be stored in the `fft_out` buffer, since the second half of the data would be equals the conjugate of the first half flipped in frequency.

That’s why the `fft_out` buffer does not need to be bigger. Attention the `fft_in` buffer will be manipulated by the FFT function and will not hold the time signal anymore after the function call.

### 2.2.4. Test Signal

You can easily check the FFT function by means of test signals even before you have put the ADC into operation.

The test signal can be e.g. a DC value (all 1s) or a sine or a complex signal like  $e^{j\phi} = \cos(\phi) + j\sin(\phi)$ .

You can either calculate the test signal in a function and fill it into the array or alternatively calculate it in Matlab, save it as CSV file and include it with #include.

## 2.3. Some useful CMSIS Functions

Besides the FFT function, CMSIS has some useful “array” functions that are also implemented very efficiently.

Some of them are e.g.

`arm_cmplx_mag_f32`

`arm_scale_f32`

`arm_add_f32`

`arm_max_f32`

If you want to compare how long a function takes, you can e.g. set a pin before execution and clear it afterwards and measure the time with an oscilloscope or by reading and comparing a timer value.

## 2.4. Some possible errors

### 2.4.1. `__FPU_PRESENT`

```
../Drivers/CMSIS/Include/core_cm4.h:105:8: error: #error "Compiler generates FPU instructions for  
a device without an FPU (check __FPU_PRESENT)"  
#error "Compiler generates FPU instructions for a device without an FPU (check __FPU_PRESENT)"  
^~~~~
```

Possible fix:

- ⇒ include "stm32f4xx\_hal.h" before you include "arm\_math.h"
- ⇒ see also <https://community.st.com/s/question/0D50X0000At139aSQA/using-dsp-libraries-eg-armmathh-in-stm32cubeide-stm32f4>

### 2.4.2. Hard Fault when calling `arm_cfft_f32`

For the complex FFT:

Check that your inout buffer size is twice the fft size (array of complex numbers,  
`cfft_inout[2*FFT_SIZE]`)

## References

- [1] CMSIS DSP Software Library, [Link](#) (2025).
- [2] ST Applicatiuon Note AN4044, “Floating point unit demonstration on STM32 microcontrollers”, DocID022737 Rev 2, May 2016. [Link](#) (2025).
- [3] Th. Lorensen, «The DSP capabilities of ARM® Cortex®-M4 and Cortex-M7 Processors, DSP feature set and benchmarks», ARM, White Paper, November 2016. [Link](#) (2025).